

# Transformer Intuition — Part 1

Word2Vec Deep Dive · Problems · Why Transformers

## ❖ 1. Word2Vec — Quick Recap

Word2Vec learns vector representations **from data** through a neural network.

Similar words → similar vectors. This was the first method to capture meaning.

king → [0.50, 0.21, -0.12, ...]

queen → [0.48, 0.19, -0.11, ...] ← close to king

dog → [0.91, -0.44, 0.87, ...] ← far from king

**Famous:** king – man + woman ≈ queen

## Vocabulary & Window Size

**Vocabulary** = collection of unique words. Case sensitive — 'Apple' ≠ 'apple'.

**Window size** = how many surrounding words are used as context during training.

Larger window → more context captured. Too large → noise increases.

Sentence: I love machine learning

Window=2, Center='love': context=['I', 'machine']

## ❖ 2. CBOW vs Skip-gram

## CBOW

Continuous Bag of Words

Predict **center word** from surrounding context words

Context: [I, machine]

Predict: 'love'

Faster to train

Better for frequent words

## Skip-gram

Predict **context words** from center word

Center: 'love'

Predict: [I, machine]

Slower to train

Better for rare words & small datasets

## ❖ 3. Problems with Word2Vec

### Problem 1 — Static Embeddings

#### The bank problem

"I sat by the river **bank**"

"I deposited money in the **bank**"

Both sentences → **same vector for 'bank'**

Word2Vec gives every word **one fixed vector** — trained once, never changes.

It cannot understand that 'bank' means different things in different contexts.

**This is called a Static Embedding.**

Transformers solved this with **Contextual Embeddings** — same word, different vector per context.

## Problem 2 — Out of Vocabulary (OOV)

Word2Vec is trained on a fixed vocabulary at a fixed point in time.

Example: Word2Vec trained in 2013.

New word: **'ChatGPT'** — not in 2013 vocabulary.

Result: Word2Vec has **no vector for it**. It simply cannot handle new words.

Transformers use **subword tokenization** — breaking unknown words into known pieces.

'ChatGPT' → ['Chat', 'G', 'PT'] → each piece has a vector → combined.

## Problem 3 — Cannot Handle Subwords

Word2Vec treats every word as atomic — no understanding of word structure.

run, running, runs, runner → 4 completely separate vectors

No shared meaning despite sharing the root 'run'.

Modern tokenizers understand morphology — 'running' = 'run' + 'ing'.

## ❖ 4. Why Transformers Were Needed

**The paper: 'Attention is All You Need' (2017)**

Introduced the Transformer architecture that all modern LLMs are built on.

Key insight: instead of processing words one by one (like RNNs), the Transformer

looks at **all words simultaneously** and figures out which words are most

relevant to each other. This is called **Self-Attention**.

## The Concept Stack — How It All Connects

One-Hot / TF-IDF → numbers, no meaning

↓

Word2Vec → static meaning (one vector per word)

↓ problem: 'bank' = same vector always

Transformers → contextual meaning (different vector per context)

↓ uses attention to understand context

LLMs (GPT, BERT) → built entirely on Transformer architecture

↓

SOTA Embeddings → what RAG and Vector DBs use today

*Note: Even the original 'Attention is All You Need' paper used Word2Vec*

*internally to convert tokens to embeddings — before passing them to the attention mechanism. Each concept builds on the previous one.*

## ❖ 5. What You Need as an AI Engineer

### For Interviews

Explain how Transformers work

Explain self-attention concept

Know encoder vs decoder

Know BERT vs GPT difference

Math = NOT required

Concept = REQUIRED

### For Building

Call pre-built embedding APIs

Google / OpenAI embedding models

sentence-transformers library

Store in Vector DB → search

Implement attention = NOT needed

Use the model = NEEDED

## ❖ 6. Quick Reference

**Word2Vec** = static embeddings, one vector per word, trained once

**CBOW** = predict center word from context (faster)

**Skip-gram** = predict context from center word (better for rare words)

**Static embedding** = same vector for 'bank' regardless of context (Word2Vec)

**Contextual embedding** = different vector based on sentence context (Transformers)

**OOV problem** = word not seen during training → no vector available

**Subword fix** = break unknown words into known pieces → each gets a vector

**Window size** = number of context words used during Word2Vec training

**Vocabulary** = set of unique words — case sensitive

**Self-Attention** = Transformer mechanism that checks how relevant

each word is to every other word in the sentence